

Residual Reinforcement Learning for Speed Control of a BDC Motor with Vertical Rod: Design, Training and Evaluation

Abstract

We add a learned, bounded correction on top of a classical speed controller for a brushed-DC (BDC) motor driving a vertical rod. A policy trained with Soft Actor-Critic (SAC) outputs a residual voltage ΔV that the controller sums with a baseline command and clips to the supply limit. We describe the plant, the control law, the learning problem (state, action, and the reward function used for training), the policy network (a multilayer perceptron, *not* a convolutional network), the training hyperparameters and their origin, how we build the train and test sets, and a comparison against PI and feedforward controllers using both machine-learning and control metrics. On a pure-PI baseline the residual learns from data the role of the feedforward and gravity-compensation terms and improves tracking, though it does not fully match the analytical controller. Network depth does not matter, and the policy does not overfit.

1. Introduction

A BDC motor spins a rigid rod; we want the shaft to track a target angular speed $\omega^*(t)$. Classical control already does this well: a PI loop plus feedforward terms. We study *residual reinforcement learning* (RL): keep the trusted classical controller and let a small neural policy add a corrective voltage on top of it. *In plain terms: the classical controller does most of the driving, and a learned “assistant” nudges the voltage to fix what the classical part misses.*

This keeps the safety and stability of the baseline while the policy learns, from interaction and with *no knowledge of the motor parameters*, the part that is hard to model. We provide a complete, reproducible pipeline (environment, training, benchmark, multi-metric evaluation) and a clear account of where residual RL helps and where it does not.

2. Plant model

The motor and rod form two coupled subsystems. The armature circuit (Kirchhoff’s voltage law) is

$$L \frac{di}{dt} = V - Ri - K_b \omega, \quad (1)$$

and the torque balance on the shaft with a uniform rod of mass m , length ℓ , centre of mass at $\ell/2$ and inertia

$$I_{\text{rod}} = \frac{1}{3}m\ell^2 \text{ is}$$

$$J_{\text{eff}} \frac{d\omega}{dt} = K_t i - B\omega - mg\frac{\ell}{2} \sin \theta, \quad \dot{\theta} = \omega, \quad (2)$$

with $J_{\text{eff}} = J_m + I_{\text{rod}}$. The gravity term $mg\frac{\ell}{2} \sin \theta$ is an angle-dependent disturbance. We integrate the equations with a fixed-step RK4 scheme at $\Delta t = 10^{-3}$ s. Parameters (Table 1) come from the Simulink multibody model (`matlab/param.m`); the converter hard-limits the supply voltage to $\pm V_{\text{max}}$.

Table 1: Plant and baseline-gain parameters.

Sym	Value	Sym	Value
R	3.0Ω	J_m	$7.4 \times 10^{-5} \text{ kg m}^2$
L	4 mH	B	$5 \times 10^{-3} \text{ N m s/rad}$
K_t	0.05 N m/A	m	0.05 kg
K_b	0.05 V s/rad	ℓ	0.1 m
V_{max}	12 V	g	9.81 m/s^2
K_p	5	K_i	2

3. Classical baselines

Three classical controllers serve both as the layer the residual sits on and as comparison targets. All output a voltage clipped to $\pm V_{\text{max}}$.

PI. A proportional-integral law on the speed error $e = \omega^* - \omega$,

$$V_{\text{PI}} = K_p e + K_i \int e dt, \quad (3)$$

with anti-windup: we clamp the integral so that $K_i \int e dt$ never exceeds $\pm V_{\text{max}}$.

Gravity feedforward. From the steady state of (1)–(2), the voltage needed to hold the rod against gravity is $V_{\text{grav}} = Rmg\frac{\ell}{2} \sin \theta / K_t$, added to the command.

Polynomial feedforward. We measure the steady-state map $\omega \mapsto V$ and invert it as a zero-bias polynomial $V_{\text{poly}}(\omega^*)$, so the controller pre-computes the bulk of the voltage. The strongest classical controller is *PI + poly + gravity*.

4. Reinforcement learning framework

Reinforcement learning (RL) learns a decision strategy by interaction rather than from labelled examples. An *agent* observes the environment state s_t , takes an action $a_t \sim \pi_\theta(\cdot | s_t)$ from its policy, and the environment returns a next state s_{t+1} and a scalar reward r_{t+1} . The agent maximises the expected discounted return

$$\max_{\theta} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_{t+1} \right], \quad \gamma \in (0, 1). \quad (4)$$

In plain terms: the policy tries an action, sees the result, and is rewarded or penalised; over many trials it learns the strategy that pays off most. Here the environment is the motor, rod and baseline controller; the agent is the residual policy; the action is the residual voltage ΔV ; and the reward (Eq. 7) rewards accurate tracking and lightly penalises the residual. The same template recurs in power-electronic control: for a DC–DC converter the state is $[V_{\text{ref}} - V_o, i_L, V_{\text{in}}]$, the action is the duty ratio, and the reward is $-(|e| + \lambda \Delta i_L^2 + \mu P_{\text{loss}})$. Table 2 lists these elements for our motor.

Table 2: The reinforcement-learning elements in our system.

RL element	In our system
Agent	residual policy π_θ (the SAC actor)
Environment	motor + rod + baseline PI controller
State s_t	normalised measurements, Eq. (6)
Action a_t	residual voltage $\Delta V = \delta a$
Reward r_t	tracking + effort + rate, Eq. (7)
Policy $\pi_\theta(a s)$	MLP mapping state to action
Return	discounted reward sum, Eq. (4)

Where a learned block sits in the loop

A neural network can enter a control loop in a few standard ways: *direct* control, where the network is the controller, $u = f_\theta(r, y, e)$; *indirect* control, where a network first identifies the plant, $\hat{y}(k+1) = f_\theta(y(k), u(k), \dots)$, and a controller (for example a predictive controller) then uses that model; and *hybrid* control, where a trusted classical controller is kept and a network either tunes its gains or adds a corrective term, $u = u_{\text{PID}} + u_{\text{NN}}$. Our method is the last case: the policy is a *compensator*, not the main controller, and it is learned by RL. *In plain terms: the classical loop does the driving; the learned part only adds a small correction, so safety and stability come from the controller we already trust.* This is the conservative choice for safety-relevant systems: keep the physical limits explicit, validate against the classical baseline, and let the learned term correct only what the baseline cannot model.

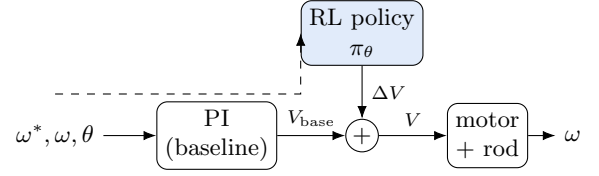


Figure 1: Residual control. A pure PI loop drives the motor; the policy observes the state and adds a bounded ΔV that supplies the feedforward and gravity-compensation correction. The controller clips the sum to $\pm V_{\text{max}}$ before it reaches the motor.

Reward design

The penalised-tracking reward used in drive and converter control has the general form

$$r = -(\text{tracking error}) - \lambda (\text{effort}) - \mu (\text{rate}).$$

The tracking error admits an absolute (L_1 , $|e|$) or a squared (L_2 , e^2) penalty; both are standard. The squared penalty punishes large transient errors more strongly, while the absolute penalty weights small persistent errors more and is robust to outliers. We use the absolute form (Eq. (7)); on this problem it lowered the residual-PI trajectory RMSE from 0.172 (L_2) to 0.081. Two checks confirm the design: weighting the error alone (α) leaves the result unchanged, because with the small effort and rate penalties it only rescales an already-dominant term, whereas raising those penalties trades tracking accuracy for a smaller, smoother residual.

5. Residual control law

The policy does not command the full voltage; it adds a *bounded residual* to the baseline and the controller hard-limits the sum:

$$V = \text{clip}(V_{\text{base}} + \Delta V, -V_{\text{max}}, V_{\text{max}}), \quad \Delta V = \delta \pi_\theta(s), \quad (5)$$

with $\pi_\theta(s) \in [-1, 1]$ and residual authority $\delta = 0.3 V_{\text{max}} = 3.6 \text{ V}$. The baseline runs every 1 ms; the policy acts at 100 Hz (once every ten steps) and holds ΔV between updates, mirroring a real digital controller and keeping the learning horizon short. Figure 1 shows the loop. *In plain terms: the AI can only push the voltage up or down by at most 3.6 V around what the classical controller already asks, and the final value can never exceed the 12 V the hardware allows.*

6. The learning problem (MDP)

We cast the task as a Markov decision process.

State. A normalised vector

$$s = \left[\frac{e}{e_s}, \frac{\omega}{\omega_s}, \sin \theta, \cos \theta, \frac{\omega^*}{\omega_s}, \frac{V_{\text{base}}}{V_{\text{max}}} \right], \quad (6)$$

The six entries are the speed error $e = \omega^* - \omega$, the measured speed ω , the rod angle through $\sin \theta$ and $\cos \theta$, the target speed ω^* , and the baseline command V_{base} . The angle enters as $(\sin \theta, \cos \theta)$ rather than θ so the representation is smooth and 2π -periodic (no jump at $\pm\pi$), and it tells the policy the angle-dependent gravity load. Passing V_{base} is what makes the policy *residual*: it sees what the baseline already commands and learns only the correction. Each entry is divided by a scale constant so every feature is of order one, which helps the network train: $\omega_s = 15$ rad/s sits just above the maximum reference (12 rad/s), and $e_s = 5$ rad/s is a moderately large error, so a typical error maps to a fraction of one and a large error to about one.

Action. A scalar $a \in [-1, 1]$, mapped to $\Delta V = \delta a$.

Reward (the training function). Per control step,

$$r = -\alpha \frac{|e|}{e_s} - \lambda \left(\frac{\Delta V}{V_{\text{max}}} \right)^2 - \mu \left(\frac{\Delta V - \Delta V_{\text{prev}}}{V_{\text{max}}} \right)^2, \quad (7)$$

The three terms are:

- **tracking**, $-\alpha |e|/e_s$: the main objective, the mean absolute speed error $|e|$ (averaged over the ten 1 ms sub-steps), normalised by e_s and weighted by α ;
- **effort**, $-\lambda (\Delta V/V_{\text{max}})^2$: keeps the residual small, so the baseline stays in charge;
- **rate**, $-\mu ((\Delta V - \Delta V_{\text{prev}})/V_{\text{max}})^2$: keeps the residual smooth, where ΔV_{prev} is the residual applied at the *previous* control step (10 ms earlier); the term penalises how much the correction jumps between consecutive updates.

Dividing by e_s and V_{max} makes the three terms dimensionless and comparable; we use $\alpha = 1$, $\lambda = 0.01$, $\mu = 0.05$. *In plain terms: the policy is paid for keeping the speed on target and fined for a large or jerky correction.* The reward never exceeds zero and uses the true (not noisy) speed, which is legitimate in simulation. Episodes last 5 s with randomised references over $[-12, 12]$ rad/s.

7. Policy network: an MLP, not a CNN

The policy and the SAC critics are *multilayer perceptrons* (MLPs): a single fully-connected hidden layer of 64 units with ReLU activations, mapping the 6-dimensional state (6) to the scalar action through a tanh squashing that guarantees $a \in [-1, 1]$. The architecture is motivated by the depth ablation in Section 11: a [64] network outperforms deeper ones and is the most consistent across seeds, so the residual is a simple, low-dimensional function.

A convolutional network (CNN) would be the wrong tool here: CNNs exploit *spatial* structure in grid-like data such as images, sharing small filters across pixels. Our input is a short, unordered vector of physical quantities, with nothing to convolve and no translation invariance to exploit. *In plain*

terms: a CNN is for pictures; our input is six numbers, so a plain fully-connected network is the right choice.

8. Training algorithm and hyperparameters

We use Soft Actor-Critic (SAC, Stable-Baselines3), an off-policy actor-critic method for continuous actions that is sample-efficient and robust. Hyperparameters (Table 3) are the SAC defaults except for the network size and the control-specific quantities; the plant gains K_p, K_i come from the classical design and the motor constants from system identification (Table 1). We train with measurement noise for robustness and evaluate noise-free for a fair, deterministic comparison.

Output. Training returns a deterministic policy $\pi_\theta : s \mapsto a$. At run time we wrap it into a controller exposing the same `step()` interface as the classical ones (Eq. 5); in operation the output is the residual voltage ΔV and tighter speed tracking.

Table 3: Training and control hyperparameters.

Parameter	Value / source
algorithm	SAC (Stable-Baselines3)
policy / critic net	MLP [64], ReLU
learning rate	3×10^{-4} (SB3 default)
discount γ	0.99
replay buffer	2×10^5
batch size	256
learning starts	5,000 steps
entropy coeff.	auto
total timesteps	3×10^5
control rate	100 Hz (decimation 10)
residual limit δ	$0.3 V_{\text{max}} = 3.6$ V
reward α, λ, μ	1, 0.01, 0.05
e_s, ω_s	5, 15 rad/s
episode length	5 s, $\Delta t = 10^{-3}$ s

9. Train and test sets

Because the policy trains on a *distribution* of references rather than a fixed dataset, we define the evaluation sets explicitly:

- **Train set:** 12 references sampled (fixed seed) from the training distribution: constant levels, smooth quintic ramps, and multi-step profiles over $[-12, 12]$ rad/s.
- **Test set:** 7 held-out references the policy never saw during selection, including negative setpoints, a multi-step profile, and one setpoint *beyond* the training range (14 rad/s, extrapolation).

Comparing train- and test-set metrics measures the generalisation gap: no gap means no overfitting.

10. Metrics: machine learning and control

This is a *regression / continuous-control* problem, so classification metrics (accuracy, F1) do not apply; there are no classes. We report, for the tracking error $e = \omega - \omega^*$:

- **MAE, MSE, RMSE:** average error magnitudes (RMSE is the primary control metric);
- R^2 : variance of the reference explained by the achieved speed, reported only for time-varying references (undefined for constant ones);
- **in-band %:** fraction of time with $|e| < 0.5$ rad/s, the honest analogue of “accuracy”;
- **parity plot** (achieved vs. reference speed, the regression analogue of a prediction-accuracy plot) and the **error autocorrelation** (a whiteness check: structure left in the error means exploitable bias remains).

11. Results

Tracking RMSE. Table 4 compares all controllers on the two reference profiles of the original study. The residual cuts the pure-PI trajectory error by about 70% but does not reach the strong classical baseline.

Table 4: RMS tracking error (rad/s). Best in green.

Controller	Const. ref	Trajectory
PI	0.495	0.322
PI + grav FFW	0.450	0.279
PI + poly FFW	0.376	0.171
PI + poly + grav	0.322	0.033
residual-PI	0.330	0.081

Multi-metric, held-out. Table 5 reports the mean over nine references. All metrics agree: residual-PI improves pure PI (in-band 85% $\rightarrow$ 99.8%), while the strong baseline stays ahead.

Table 5: Mean metrics over 9 references (train+test).

Controller	MAE	RMSE	R^2	in-band
PI	0.281	0.410	0.988	85.2%
PI + poly + grav	0.046	0.212	0.997	99.8%
residual-PI	0.078	0.233	0.997	99.8%

Generalisation. On the seven held-out references, residual-PI beats pure PI on *all* of them including the extrapolated 14 rad/s setpoint (mean RMSE 0.240 vs 0.411), so the gain holds beyond the two reported references; the held-out mean is now close to the classical PI + poly + gravity (0.222) though still behind.

Overfitting. Table 6 compares the base network on train vs. test: the test error is no worse than train, so the policy does not overfit (if anything the sampled training references are harder).

Table 6: Base network (MLP [64]): train vs. test.

Set	MAE	RMSE	R^2	in-band
train	0.069	0.292	0.955	99.7%
test	0.080	0.240	0.996	99.8%

Network depth. Table 7 sweeps the hidden-layer size over three seeds. The smallest [64] network is the best and the most consistent (per-seed std ≤ 0.004), so the multi-seed result confirms the L1 gain is robust rather than seed-luck; *depth does not help*, because the residual is a simple function.

Table 7: Depth ablation, test set (mean $\pm$ std, 3 seeds).

net_arch	RMSE	R^2	in-band
[64]	0.238 $\pm$ 0.001	0.996	99.8%
[256, 256]	0.256 $\pm$ 0.004	0.995	99.8%
[256, 256, 256]	0.247 $\pm$ 0.001	0.996	99.8%

Figures 2–4 show the closed-loop tracking, the parity plots, and the depth ablation.

12. Saturation handling

The hard clip in (5) enforces the supply limit on the *total* voltage before the motor; the plant itself does not clip. The PI integrator has anti-windup, and the residual is stateless, so no windup arises there. The voltage sits at the ± 12 V rail only 0.1–0.4% of the time (brief transients), so saturation does not limit performance in the present regime.

For operation at higher speeds, where saturation becomes frequent, a *saturation-aware* mode is available: the state (6) gains the previous applied (clipped) voltage $V_{\text{applied}}/V_{\text{max}}$, and the reward (7) penalises the *effective* (post-clip) residual plus a clip-excess term $-\nu(\max(0, |V_{\text{base}} + \Delta V| - V_{\text{max}})/V_{\text{max}})^2$, so the policy senses the rail and avoids wasting effort against it.

13. Conclusion

A residual SAC policy on top of a pure PI loop, trained with no parameter knowledge, learns from data the role of the feedforward and gravity-compensation terms: it lowers the trajectory RMSE by about 75% and generalises to unseen and extrapolated references. It does not fully match the analytical PI + poly + gravity controller (0.081 vs 0.033 rad/s), so the learned substitute is partial. A single hidden layer of 64 units is the right network, depth is irrelevant, and the policy does not overfit. Residual RL is worth it when you lack a usable model; check any improvement on held-out

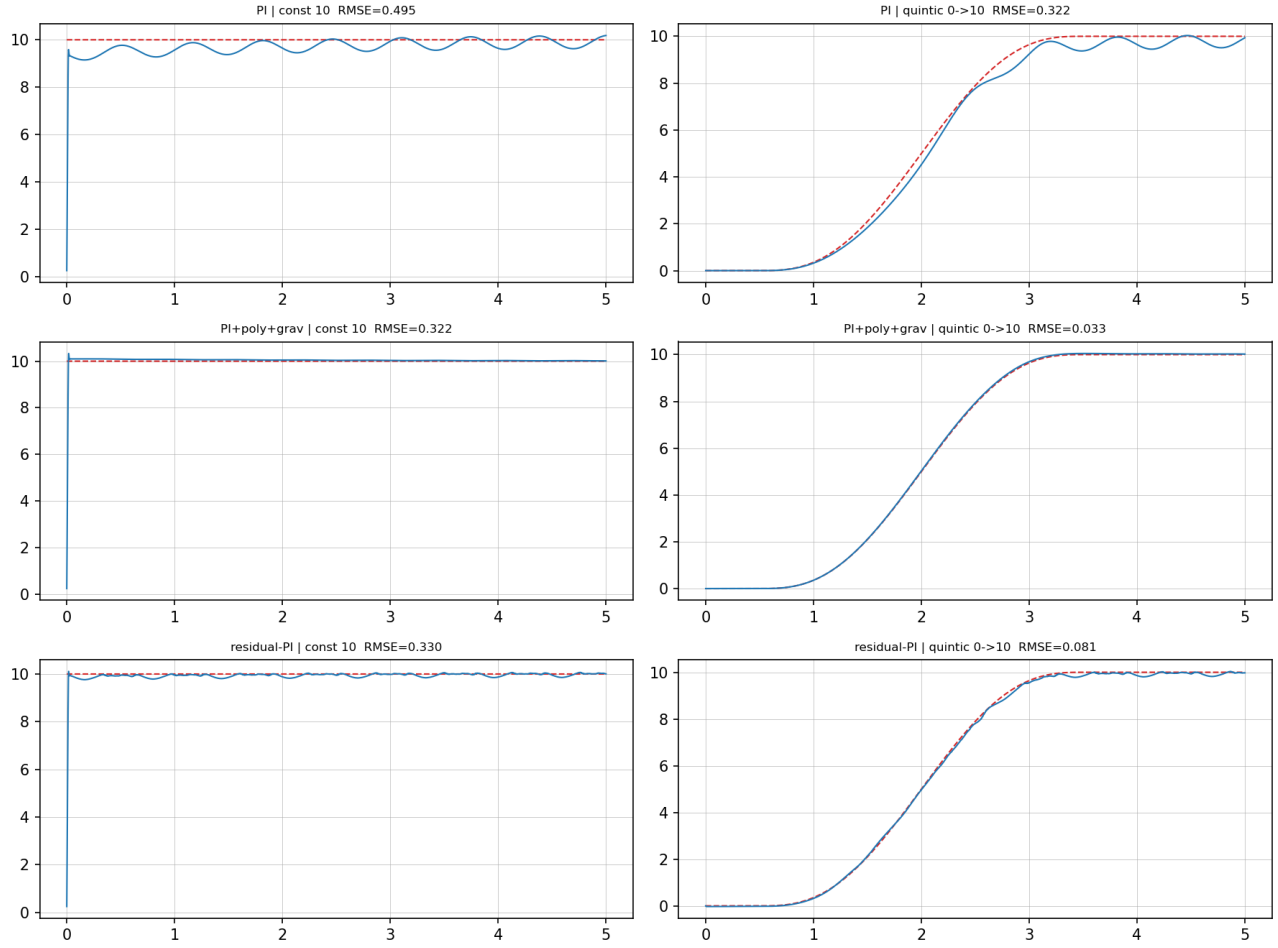


Figure 2: Closed-loop speed tracking (blue) vs. reference (dashed red) for each controller on the two reference profiles.

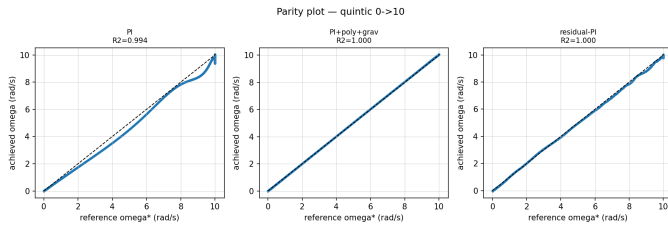


Figure 3: Parity plot (achieved vs. reference speed) on the quintic trajectory; points on the diagonal mean perfect tracking.

references, across seeds, and with metrics suited to a control problem.

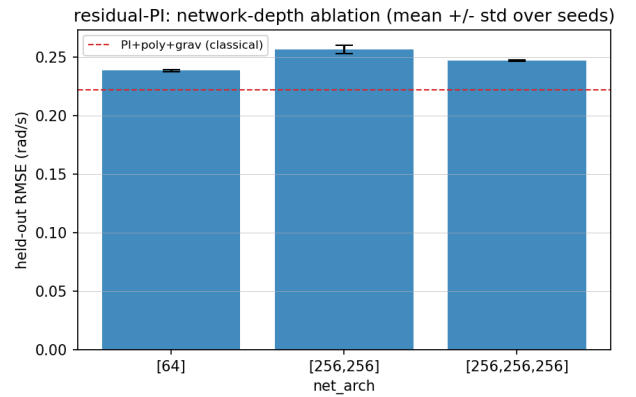


Figure 4: Network-depth ablation: held-out RMSE (mean $\pm$ std over seeds); the red line is the strong classical baseline.